

New

Home

Learn

Workspace

Recent

Catalog

Jobs & Pipelines

Compute

Discover Beta

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Spaces

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

AI/ML

Playground

AI Gateway

Experiments

Features

Modes

Serving

01\_bronze\_ingesta

File Edit View Run Help Python Last edit was 38 minutes ago

Run all Serverless Schedule Share

Last execution failed

Python

```
1 # databricks notebook source
2
3 # COMMAND -----
4 # Configuración
5
6 CATALOG_NAME = "sparkspace"
7 SCHEMA_NAME = "default"
8 TABLE_NAME = "bronze_retail_sales"
9 FULL_TABLE_NAME = f"{CATALOG_NAME}.{SCHEMA_NAME}.{TABLE_NAME}"
10 SOURCE_CSV_PATH = "Volumes/workspace/default/raw_files/retail_store_sales.csv"
11
12 # COMMAND -----
13 # Verificación de schema
14
15 spark.sql(f"CREATE SCHEMA IF NOT EXISTS {CATALOG_NAME}.{SCHEMA_NAME}")
16
17 # COMMAND -----
18 # Verificación de archivo fuente
19
20 import os
21
22 files = dbutils.fs.ls(os.path.dirname(SOURCE_CSV_PATH))
23 found = [f.name for f in files if f.name == os.path.basename(SOURCE_CSV_PATH)]
24 if not found:
25     raise FileNotFoundError(f"Archivo no encontrado: {SOURCE_CSV_PATH}")
26 print(f"✓ Archivo encontrado: {SOURCE_CSV_PATH}")
27
28 # COMMAND -----
29 # Lectura CSV
30
31 import pandas as pd
32
33 pdf = pd.read_csv(SOURCE_CSV_PATH)
34 print(f"Filas: {len(pdf)}, Columnas: {list(pdf.columns)}")
35
36 # COMMAND -----
37 # Metadatos técnicos
38
39 from datetime import datetime, timezone
40
41 ingestion_ts = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%S")
42 pdf["ingestion_timestamp"] = ingestion_ts
43 pdf["source_file"] = SOURCE_CSV_PATH
44 pdf["source_row_count"] = len(pdf)
45
46 # COMMAND -----
47 # Escritura Bronze
48
49 sdf = spark.createDataFrame(pdf)
50
51 {
52     sdf.write
53     .format("delta")
54     .mode("overwrite")
55     .option("overwriteSchema", "true")
56     .saveAsTable(FULL_TABLE_NAME)
57 }
58
59 print(f"✓ Tabla escrita: {FULL_TABLE_NAME}")
60
61 # COMMAND -----
62 # Validaciones
63
64 count_csv = len(pdf)
65 count_delta = spark.table(FULL_TABLE_NAME).count()
66
67 print(f"Filas CSV : {count_csv}")
68 print(f"Filas Delta : {count_delta}")
69
70 assert count_delta == count_csv, f"Diferencia: {abs(count_delta - count_csv)} filas"
71 print(f"✓ Conteo coincide")
72
73 display(spark.table(FULL_TABLE_NAME).limit(5))
74
75 spark.sql(f"DESCRIBE TABLE EXTENDED {FULL_TABLE_NAME}").show(truncate=False)
76
77 > See performance @
78
79 ✓ Archivo encontrado: Volumes/workspace/default/raw_files/retail_store_sales.csv
80 Filas: 12,575 | Columnas: ['Transaction ID', 'Customer ID', 'Category', 'Item', 'Price Per Unit', 'Quantity', 'Total Spent', 'Payment Method', 'Location', 'Transaction Date', 'Discount Applied']
81
82 [DELTA_INVALID_CHARACTERS_IN_COLUMN_NAMES] Found invalid character(s) among '.,;()\\n\\t' in the column names of your schema.
83 Invalid column names: Transaction ID, Customer ID, Price Per Unit, Total Spent, Payment Method, Transaction Date, Discount Applied.
84 Please use other characters and try again.
85 Alternatively, enable Column Mapping to keep using these characters. SQLSTATE: 42K05
```

Diagnose error

Debug

Genie Code Quick Fix ON

01:03:18

29/4/2026



Error claro. Delta no acepta espacios ni caracteres especiales en nombres de columna. El fix es renombrar todas las columnas al leer el CSV, antes de crear el Spark DataFrame.

Reemplazé la celda de Escritura Bronze con esto:

```
python

# Escritura Bronze

import re

def clean_col(name):
    return re.sub(r'[^a-zA-Z0-9_]', '_', name).strip('_').lower()

pdf.columns = [clean_col(c) for c in pdf.columns]

sdf = spark.createDataFrame(pdf)

(
    sdf.write
        .format("delta")
        .mode("overwrite")
        .option("overwriteSchema", "true")
        .saveAsTable(FULL_TABLE_NAME)
)

print(f"✓ Tabla escrita: {FULL_TABLE_NAME}")
print(f"Columnas finales: {list(pdf.columns)}")
```

Lo que hace `clean_col`: convierte espacios y caracteres especiales (`' '`, `'/'`, etc.) en `_`, pasa todo a minúsculas, y elimina guiones bajos al inicio o final. Las columnas van a quedar así:

| Original         | Renombrada       |
|------------------|------------------|
| Transaction ID   | transaction_id   |
| Customer ID      | customer_id      |
| Price Per Unit   | price_per_unit   |
| Total Spent      | total_spent      |
| Payment Method   | payment_method   |
| Transaction Date | transaction_date |
| Discount Applied | discount_applied |
| Category         | category         |
| Item             | item             |
| Quantity         | quantity         |
| Location         | location         |

Esto también es consistente con el `snake_case` que usamos en Silver y Gold, así que no necesitás renombrar de nuevo en el notebook 02.

Solo reemplazé esa celda, el resto del notebook queda igual. Volvé a correr desde esa celda hacia abajo.



```
|col_name|data_type|
|comment|
+-----+-----+
|transaction_id|string|
|NULL|
|customer_id|string|
|NULL|
|category|string|
|NULL|
|item|string|
|NULL|
|price_per_unit|double|
|NULL|
|quantity|double|
|NULL|
|total_spent|double|
|NULL|
|payment_method|string|
```

The screenshot displays the Databricks interface with a SQL notebook. The code defines a schema and creates a DataFrame from a series of StructField objects. The output shows a summary of data quality checks for fact\_sales, including counts of nulls and errors for various fields like transaction\_id, customer\_key, item\_key, date\_key, and payment\_method.

```
log("Conteo fact_sales > 0", count_fact > 0, f"filas={count_fact}")
log("Conteo fact_items > 0", count_item > 0, f"filas={count_item}")
log("Conteo dia_item > 0", count_date == 1114, f"filas={count_date}")
```

# COPWAD -----  
# Table resumen final

```
from pyspark.sql.types import StructType, StructField, StringType

schema_res = StructType([
    StructField("nombre_validacion", StringType(), True),
    StructField("resultado", StringType(), True),
    StructField("detalle", StringType(), True),
])

resumen = spark.createDataFrame(results, schema=schema_res)
display(resumen)
```

See performance (22)

- silver: pyspark.sql.connect.dataframe.DataFrame = [item string, transaction\_id string ~ 12 more fields]
- fact: pyspark.sql.connect.dataframe.DataFrame = [transaction\_id string, customer\_key long ~ 8 more fields]
- d\_cust: pyspark.sql.connect.dataframe.DataFrame = [customer\_key long, customer\_id string ~ 1 more field]
- d\_item: pyspark.sql.connect.dataframe.DataFrame = [item key long, item\_id string ~ 3 more fields]
- d\_date: pyspark.sql.connect.dataframe.DataFrame = [date key integer, full\_date date ~ 8 more fields]
- resumen: pyspark.sql.connect.dataframe.DataFrame = [nombre\_validacion string, resultado string ~ 1 more field]

|  | fact_sales | fact_items | dia_item | dim_customer | dim_item | dim_date |
|--|------------|------------|----------|--------------|----------|----------|
|  | 11,362     | 11,362     | 25       | 200          | 1,114    | 1,114    |

- OK - Conteo Silver vs fact\_sales: Silver=11,362 | fact=11,362
- OK - Unicidad transaction\_id: duplicados=0
- OK - PK nula - dim\_customer.customer\_key: nulos=0
- OK - PK nula - dim\_item.item\_key: nulos=0
- OK - PK nula - dim\_date.date\_key: nulos=0
- OK - FK nula - customer\_key: nulos=0
- OK - FK nula - item\_key: nulos=0
- OK - FK nula - date\_key: nulos=0
- OK - Integridad referencial - dim\_customer: huérfanos=0
- OK - Integridad referencial - dim\_item: huérfanos=0
- OK - Integridad referencial - dim\_date: huérfanos=0
- OK - Consistencia total\_spent: filas con diff>0.02=0
- OK - Dominio payment\_method: inválidos=0
- OK - Dominio location: inválidos=0
- OK - Dominio discount\_applied (sin nulos): nulos=0
- OK - Rango fechas dim date cubre fact sales: dim=[2022-01-01..2025-01-01] fact=[2022-01-01..2025-01-01]

|    | A <sup>2</sup> : nombre_validation    | A <sup>2</sup> : resultado | A <sup>2</sup> : detalle    |
|----|---------------------------------------|----------------------------|-----------------------------|
| 1  | Conteo Silver vs fact_sales           | ✓ OK                       | Silver=11,362   fact=11,362 |
| 2  | Unicidad transaction_id               | ✓ OK                       | duplicados=0                |
| 3  | PK nula — dim_customer.customer_key   | ✓ OK                       | nulos=0                     |
| 4  | PK nula — dim_item.item_key           | ✓ OK                       | nulos=0                     |
| 5  | PK nula — dim_date.date_key           | ✓ OK                       | nulos=0                     |
| 6  | FK nula — customer_key                | ✓ OK                       | nulos=0                     |
| 7  | FK nula — item_key                    | ✓ OK                       | nulos=0                     |
| 8  | FK nula — date_key                    | ✓ OK                       | nulos=0                     |
| 9  | Integridad referencial — dim_customer | ✓ OK                       | huérfanos=0                 |
| 10 | Integridad referencial — dim_item     | ✓ OK                       | huérfanos=0                 |
| 11 | Integridad referencial — dim_date     | ✓ OK                       | huérfanos=0                 |
| 12 | Consistencia total_spent              | ✓ OK                       | filas con diff>0.02=0       |
| 13 | Dominio payment_method                | ✓ OK                       | inválidos=0                 |
| 14 | Dominio location                      | ✓ OK                       | inválidos=0                 |
| 15 | Dominio discount_applied (sin nulos)  | ✓ OK                       | nulos=0                     |

20 rows | 17.92s runtime Refreshed now